

Yes. What you are describing is essentially a **device simulator / hardware emulator**, and I think your current v1 is only partially solving the problem.

Your current `interactive_device_stub.py` is already closer to the right direction than `gateway_simulator.py`: it models **two independent terminals**, maintains users on each terminal, supports downstream CRUD, local enrollment/editing, and emits attendance events.

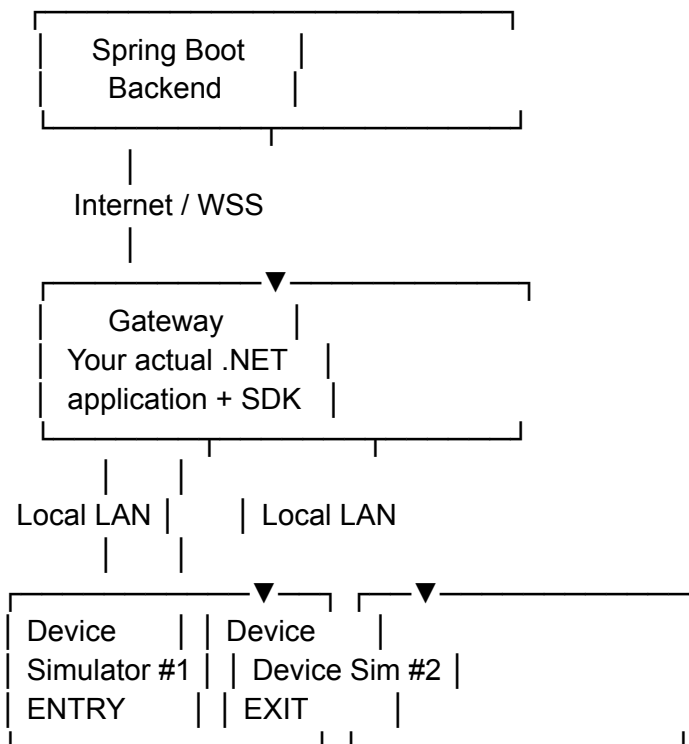
However, the important distinction is:

You don't just want to mock the Gateway → Backend protocol. You want to simulate the actual behavior of the devices behind the Gateway so that the Gateway itself is exercised almost exactly as it would be in production.

Your current simulator largely bypasses that. It talks directly to the Spring Boot backend using `/internal/gateway/messages`, rather than making the Gateway communicate with something that behaves like the real device.

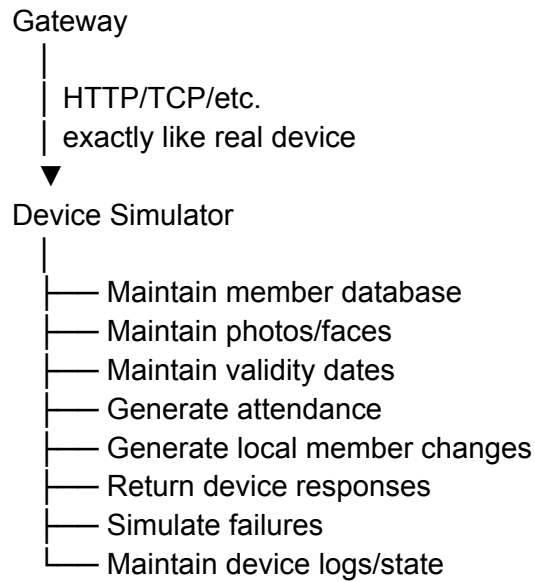
What I would build instead

I'd structure the test environment like this:

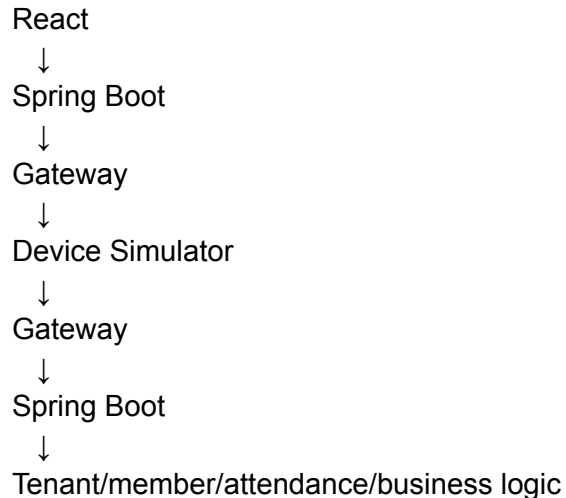


And the simulator should expose the **same device-facing API/protocol that the real hardware exposes**.

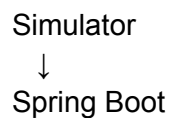
That means:



This gives you a much more valuable test:



Instead of:



which is what your current `gateway_simulator.py` effectively does. It explicitly posts directly to `/internal/gateway/messages.gateway_simulator`

Why this matters for your application

Your concern about **multiple sources of truth and complicated final-state logic** is exactly why I would invest in this.

You should be able to create scenarios such as:

Backend says:

Member 123 = active
validity = Jan 1 → Dec 31

Device says:

Member 123 = active
validity = Jan 1 → Jun 30

Local device operator changes:

Member 123 → disabled

Attendance:

Member 123 attempts entry

Network:

Gateway loses internet for 30 minutes

Gateway:

queues device changes

Internet returns:

Gateway reconnects
synchronization happens

Then observe whether your **real Gateway + backend** eventually reach the expected state.

And you can deliberately create much nastier cases:

- device offline
- gateway offline
- backend offline
- device responds slowly
- device returns an error
- device accepts command but response is lost
- duplicate command
- duplicate attendance event
- stale member update
- local device modification

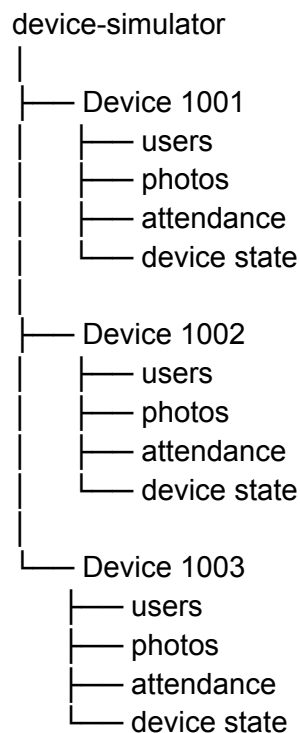
- membership expiry while offline
- photo upload failure
- partial synchronization
- Gateway restart
- device restart
- backend restart
- two devices having different state
- same member existing on one device but not another
- commands arriving out of order

Those are the cases that a simple mock usually doesn't expose.

One important change to your proposed approach

I **wouldn't necessarily make every simulated device a separate Node process.**

I'd make one **Device Simulator Server** capable of hosting multiple virtual devices:



with configuration such as:

Device 1001
IP: 127.0.0.1:9001

Device 1002
IP: 127.0.0.1:9002

or, if your Gateway discovers devices differently, replicate that mechanism.

Then your actual Gateway can be configured to believe:

Entry Device → 127.0.0.1:9001

Exit Device → 127.0.0.1:9002

The simulator can have an admin UI/API:

GET /devices

POST /devices

POST /devices/{id}/users

PUT /devices/{id}/users/{userId}

DELETE /devices/{id}/users/{userId}

POST /devices/{id}/attendance

POST /devices/{id}/simulate-offline

POST /devices/{id}/simulate-error

POST /devices/{id}/reset

This gives you a **test laboratory for the entire Gateway**.

Your existing v1

Your current files actually contain useful pieces that I'd preserve.

For example, your existing stub already models per-device user state and validity dates.
interactive_device_stub

It also already handles simulated attendance events with `deviceId`, timestamp, recognition method, grant/deny state and record number. interactive_device_stub

And it has the concept of **local device-originated changes flowing upstream**, which is important for your architecture. interactive_device_stub

So I wouldn't throw it away. I'd **refactor it from a GUI/backend mock into a proper device-protocol emulator**, and retain the GUI as an optional testing console.

The Copilot prompt should come before implementation

I agree with your approach of **not immediately asking Copilot to implement this**.

First, I would give Copilot a prompt that says essentially:

Do not modify code yet. Analyze the existing Gateway, SDK integration, device communication layer, Spring Boot backend flows, and these existing simulators. Determine the exact device-facing contract and propose the simulator architecture.

It should produce:

1. **Every device API/method currently used by Gateway**
2. Exact method signatures
3. Request/response structures
4. Success/error semantics
5. Device → Gateway callbacks/events
6. Gateway → Device commands
7. Attendance flow
8. Member creation/update/deletion flow
9. Membership validity flow
10. Face/photo flow
11. Device synchronization flow
12. Retry behavior
13. Timeout behavior
14. Offline behavior
15. Device identification
16. Multiple-device behavior
17. Which parts of the existing v1 are incorrect/incomplete
18. Proposed simulator architecture
19. Required simulator endpoints
20. Test scenarios that the simulator should support

And critically:

Do not infer a device API merely because it would be convenient. Trace it from the actual SDK/wrapper/Gateway implementation and identify anything that cannot be verified from the repository.

That last part is important. Otherwise Copilot may invent an API that looks plausible but doesn't actually match your TrueFace integration.

Your `requirements.txt` also shows that the existing simulator mixes Pillow, requests and websocket-client, so Copilot should verify whether those dependencies are still appropriate rather than blindly carrying them forward. requirements

I can create that detailed Copilot analysis prompt for you next, specifically tailored to your repository and the three files you've provided, so that Copilot first produces an architecture/contract report and **doesn't implement anything until you approve it**.